



A mini symbolic execution engine

May 26, 2015 • Xi Wang

It has been a while since I blogged last time. One change is that I started to have nightmares about me forgetting to prepare for lecture, rather than forgetting to turn in homework as in the past 20(?) years—life does get better.

This post is about teaching. Last week in grad OS class ([CSE 551](#)) we talked about symbolic execution, using the papers of [KLEE](#) and [SAGE](#). To illustrate the basic idea, I wrote a mini symbolic execution implementation (~20 lines of Python code), [mini-mc](#), using Z3.

Consider a simple example, [test_me.py](#):

```
from mc import *

def test_me(x, y):
    z = 2 * x
    if z == y:
        if y == x + 10:
            assert False

x = BitVec("x", 32)
y = BitVec("y", 32)
test_me(x, y)
```

Here `test_me()` looks like a normal Python function: the program would crash if the execution hits `assert False`. The two input variables `x` and `y` are initialized to symbolic 32-bit integers, through [Z3's python](#)

interface.

To find input that would trigger a crash with symbolic execution, one can fork at every if-branch, and explore each side of the branch. With [mini-mc](#), simply running the Python program will do so, producing the following output:

```
[3216] assume (2*x == y)
[3217] assume ¬(2*x == y)
[3217] exit
[3216] assume (y == x + 10)
[3218] assume ¬(y == x + 10)
[3218] exit
[3216] Traceback (most recent call last):
  File "./test_me.py", line 11, in <module>
    test_me(x, y)
  File "./test_me.py", line 7, in test_me
    assert False
AssertionError: x = 10, y = 20
[3216] exit
```

The output says that the execution starts with PID 3216, and forks a child process 3217 to explore the false branch of `if z == y`, while itself continues to explore the true branch. It does so similarly for a second branch `if y == x + 10`, and then hits `assert False` with input `x = 10, y = 20`.

Here's an almost complete implementation of the symbolic execution engine in [mc.py](#):

```
def sched_fork(self):
    pid = os.fork()
    if pid:
        solver.add(self)
        r = True
        mc_log("assume (%s)" % (str(self),))
```

```

else:
    solver.add(Not(self))
    r = False
    mc_log("assume ~(%s)" % (str(self),))
    if solver.check() != sat:
        mc_log("unreachable")
        sys.exit(0)
    return r
# intercept BoolRef -> bool conversions
setattr(BoolRef, "__bool__", sched_fork)
setattr(BoolRef, "__nonzero__", getattr(BoolRef, "__bool__"))

```

That's it - just one function, to show the basic fork-explore-check idea.

The trick is that, as values are symbolic (e.g., via Z3's `BitVec`), the Python VM will try to convert them into bool at `if` statements; let's intercept the conversion and replace it with `sched_fork()`, by rewriting `__bool__` (Python 3.x) or `__nonzero__` (Python 2.x).

There are two more fun examples on equivalence checking:

- [ffs_eqv.py](#): check the equivalence of two find-first-set implementations, from the UC-KLEE paper (CAV 2011); and
- [mod_eqv.py](#): check the equivalence of two modulo implementations, from the KLEE paper (OSDI 2008).

Hope this is useful for classroom demonstration.

BTW, [mc.py](#) also contains a mini implementation of concolic execution. Run [bad.py](#), which resembles [Figure 1 of the SAGE paper](#), and you will see 5 (out of 16) inputs that trigger the exception:

```

[4324] =====
[4324] #0: s[0] = 0, s[1] = 0, s[2] = 0, s[3] = 0
[4324] s[0] == 98: False
[4324] s[1] == 97: False
[4324] s[2] == 100: False

```

```
[4324] s[3] == 33: False
[4324] =====
[4324] #1: s[0] = 0, s[1] = 0, s[2] = 0, s[3] = 33
[4324] s[0] == 98: False
[4324] s[1] == 97: False
[4324] s[2] == 100: False
[4324] s[3] == 33: True
    ...
[4324] =====
[4324] #15: s[0] = 98, s[1] = 97, s[2] = 100, s[3] = 33
[4324] s[0] == 98: True
[4324] s[1] == 97: True
[4324] s[2] == 100: True
[4324] s[3] == 33: True
[4324] Traceback (most recent call last):
  File "./bad.py", line 32, in <lambda>
    mc_fuzz(lambda: top(s), s, [0] * n)
  File "./bad.py", line 26, in top
    assert False
AssertionError: s[0] = 98, s[1] = 97, s[2] = 100, s[3] = 33
[4324] exit
```

You can modify [test_me.py](#) to do the same. See the [mini-mc](#) repo on github for details.

